

PROJECT / OPENMP / IRREGULAR PARALLELISM

Adaptive quadrature with a task tree

Scientific question. When does dynamically generated numerical work justify OpenMP tasks, and how should task granularity be controlled?

Problem and core scope

Implement adaptive Simpson's integration rule to approximate

$$\int_a^b f(x) dx$$

on a finite interval $[a, b]$ and apply it to a parameter sweep of sharply peaked functions. Use

$$f_{c,\delta}(x) = \frac{1}{1 + ((x - c)/\delta)^2}, \quad 0 \leq x \leq 1,$$

whose exact integral is

$$I(c, \delta) = \delta \left[\arctan \frac{1 - c}{\delta} + \arctan \frac{c}{\delta} \right].$$

Vary peak locations and widths, for example $c \in [0.1, 0.9]$ and $\delta \in [10^{-3}, 10^{-1}]$. The core is adaptive Simpson integration, a task-parallel version, and a comparison with parallelizing whole independent integrals. Improper and singular integrals are outside scope.

Algorithmic suggestions

Given a global error τ , for a generic interval $[a_i, b_i]$, with midpoint m_i , define

$$S(a_i, b_i) = \frac{b_i - a_i}{6} \{f(a_i) + 4f(m_i) + f(b_i)\}.$$

Compute $S_2 = S(a_i, m_i) + S(m_i, b_i)$ using two new evaluations. With $D = S_2 - S(a_i, b_i)$, accept $I_i = S_2$ if $|D|/15 \leq \tau_{a,b_i} = \tau \frac{b_i - a_i}{b - a}$ and then add I_i to the variable I accumulating the result. Otherwise, recurse on both halves $[a_i, m_i]$ and $[m_i, b_i]$. Cache reused function values. You start by seeding the integration interval $[a, b]$ with a modest fixed partition, such as 16 subintervals. Error estimates can miss unsampled features; they are not guaranteed bounds for arbitrary functions. For known narrow peaks, it is better to add as initial breakpoints a few point near the peak. Validate the selected test family explicitly by computing the error between the exact integral and the approximation. Verify if the error is below τ .

Use openMP task base parallelism. Create one OpenMP `parallel` region and a `single` root. Spawn recursive tasks only above a *chosen depth/work cutoff*; below it, call the sequential recursion. Pass interval data by value, store child results separately, and use `taskwait` before combining them or returning. Parent stack objects shared with tasks must remain alive until those tasks finish.

Core output

For each integral, report estimate, actual error, estimated error, number of function evaluations, task count, maximum recursion depth, and success status. Avoid printing from worker tasks, it is inefficient.

PROJECT 5 / VALIDATION AND DELIVERY**Adaptive quadrature: checks and experiments****Validation and acceptance**

- Verify constant and cubic polynomial integrals, where the formula is exact (up to floating point error) then compare peaked integrals with the analytic formula. Use several tolerances, such as 10^{-6} , 10^{-8} , 10^{-10} .
- Include peaks lying away from the initial sample points to expose missed-feature failures. Explain how the initial partition or supplied breakpoints affect reliability.
- Compare serial and task results within the requested accuracy, and confirm matching stopping/failure behaviour. Exercise the depth-limit and floating-point interval-collapse paths.

Required performance study

Create a deterministic set of about 100 parameter pairs. Compare sequential adaptive integration with the OpenMP task implementation.

Report speedup, task count, evaluations, and time. Cheap integrands may make recursive tasking slower.

Optional extension

Make the code more general, accepting any callable object and different integration rules. For instance, you may try a 5 points Gauss integration rule (see [this page](#) or any good book on numerical methods).

As a. more complex, alternative, you may try to extend the code for the computation of two dimensional integrals

$$\int_a^b \int_c^d f(x, y) dx dy,$$

trying to reuse most of the already produced code.

Accessible starting references

- **Driscoll and Braun: Adaptive integration.** Open textbook on recursive error-controlled integration, reuse of evaluations, and the limitations of error estimates. (but you can use any good book on numerical methods)
- **Fundamentals of Numerical Computation: earlier adaptive-integration chapter.** A freely available alternative exposition of the Simpson-based divide-and-conquer approach.
- **LLNL: OpenMP tutorial.** Read TASK, TASKWAIT, SINGLE, and data-scope sections before implementing recursion.